



Application Layer: Architecture, Protocols and APIs.

CSAI321:Computer Networks

Join the lecture online on your dashboard

Lecture Roadmap

- Network Architectures
- Application Layer
- HTTP - The Web's Language
- HTTP Methods Overview
- The Evolution of HTTP
- Head-of-Line (HoL) Blocking
- HTTP2.0 and HTTP3.0
- REST Principles

Network architecture

Network architecture

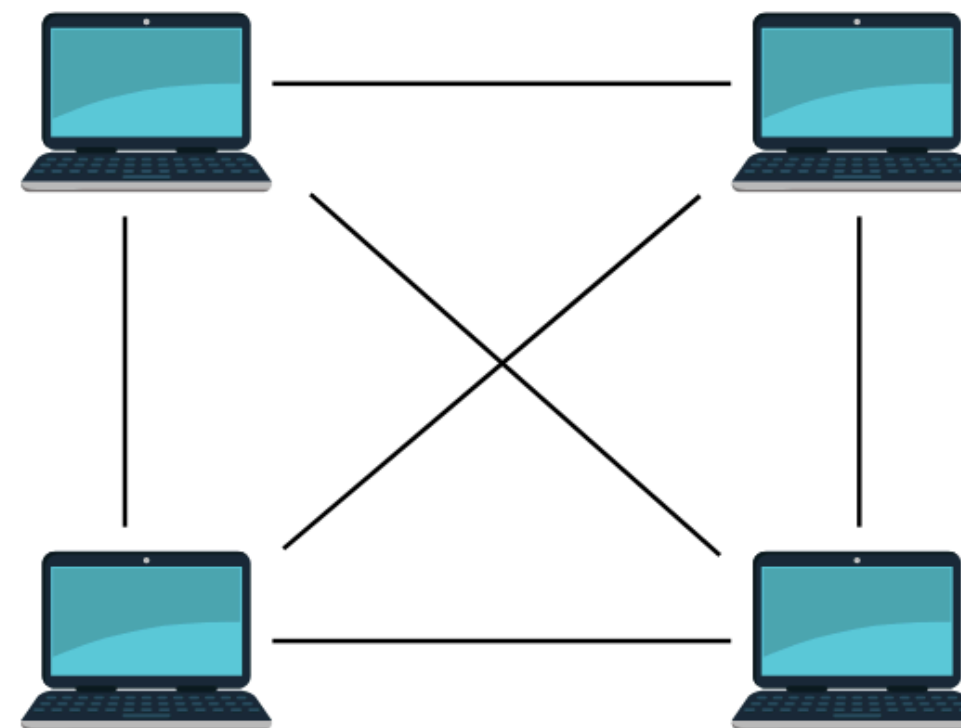
Client-Server Architecture



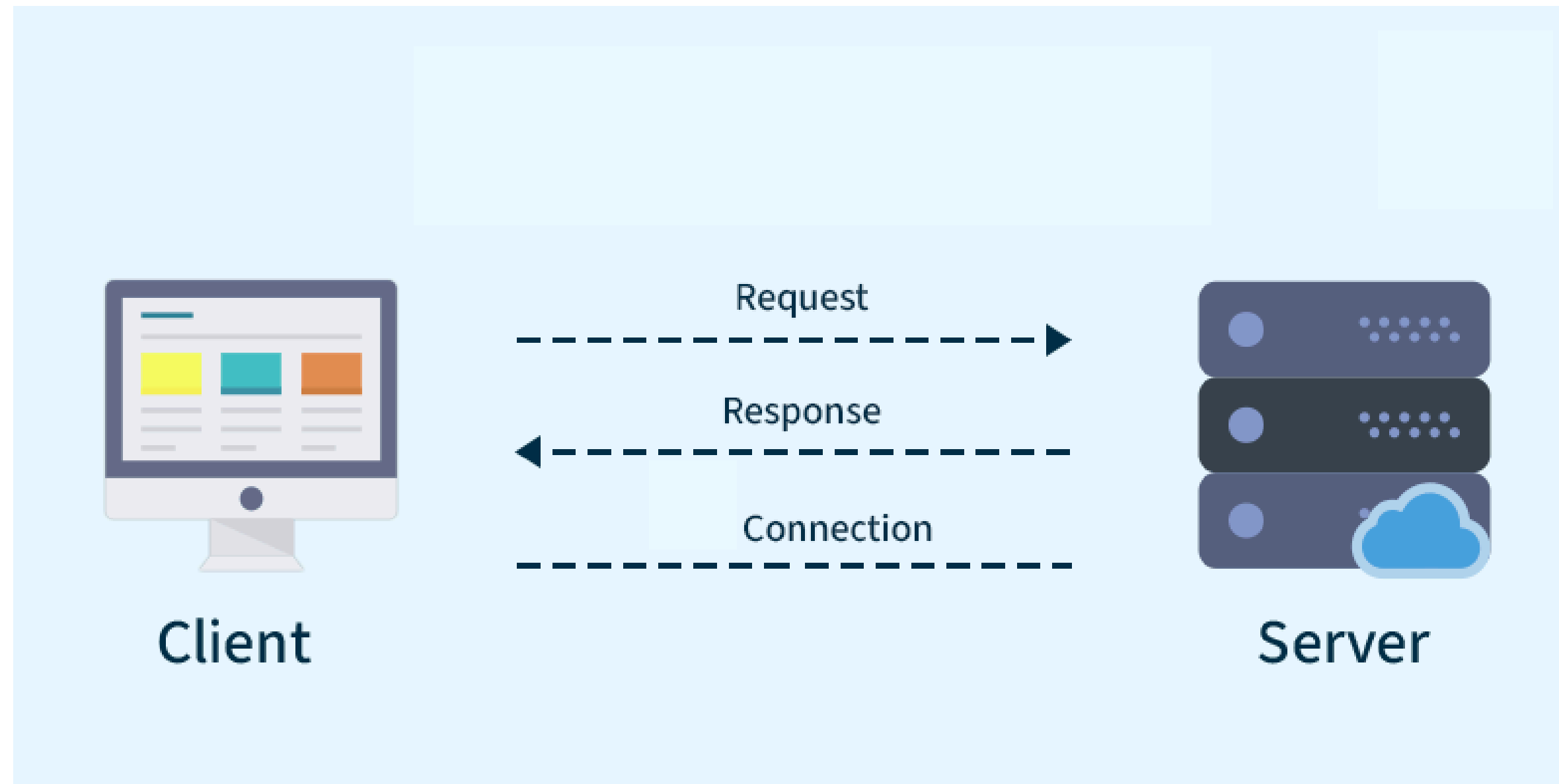
- **Client/Server:** Centralized architecture where powerful servers store data and process requests for multiple dependent clients.

Network architecture

Peer-to-Peer (P2P) Architecture



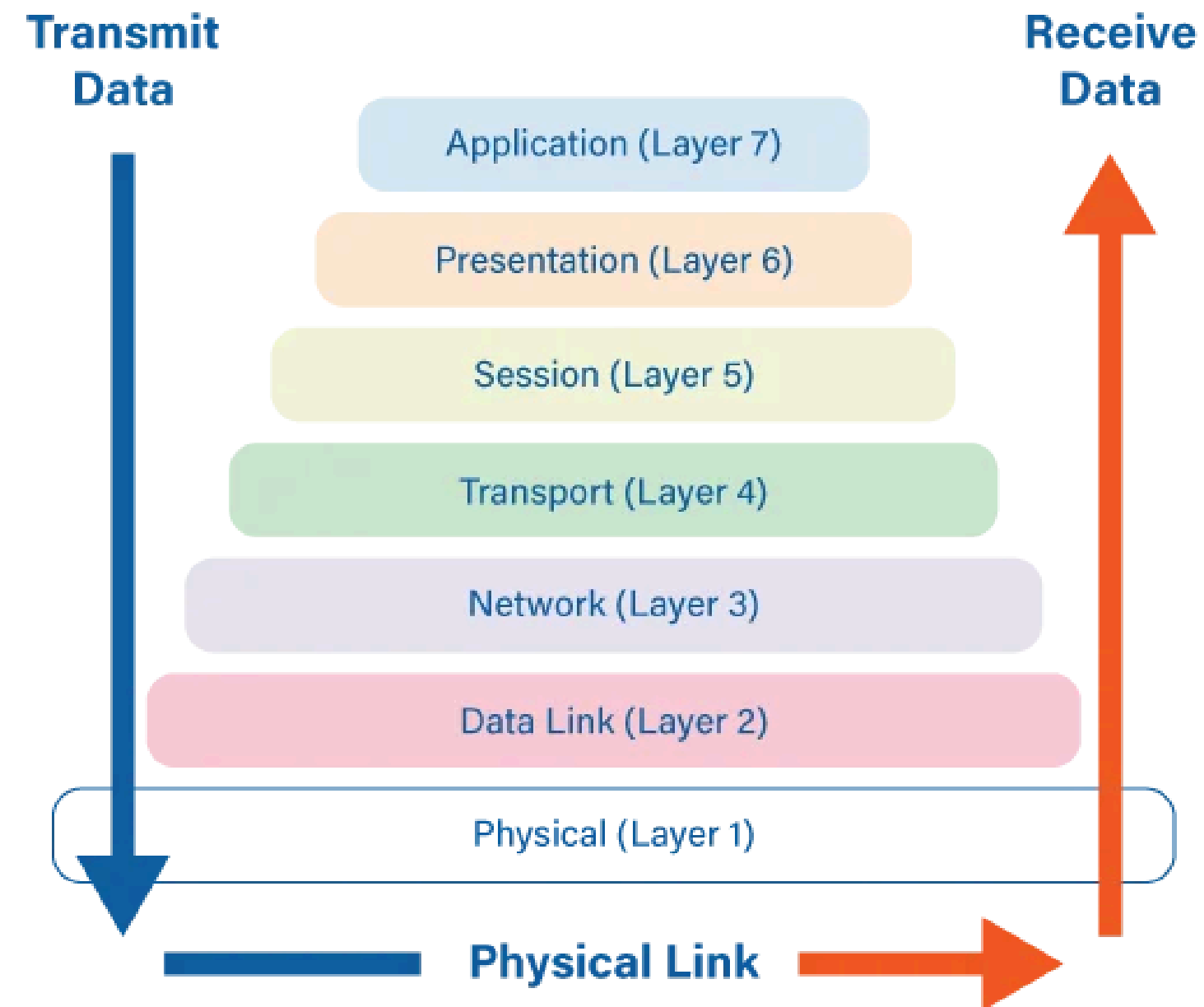
- **Peer-to-Peer (P2P):** Decentralized model where all devices ("peers") act as both clients and servers with equal privileges



How do these systems actually talk to each other over the network?

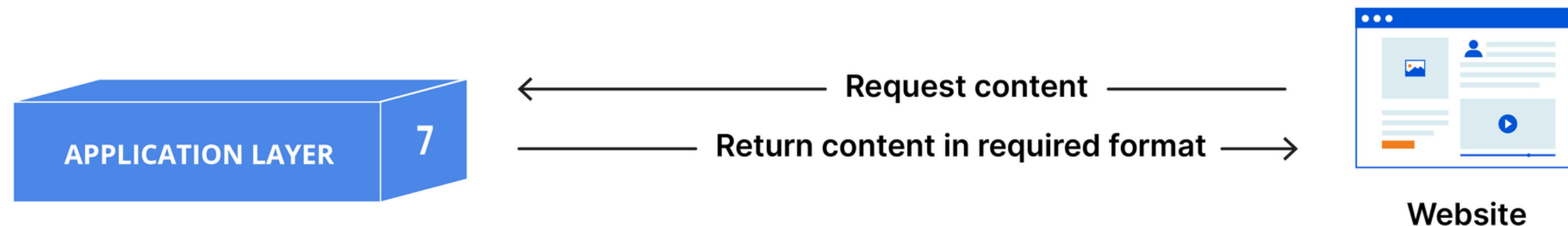
Recap

The 7 Layers of OSI



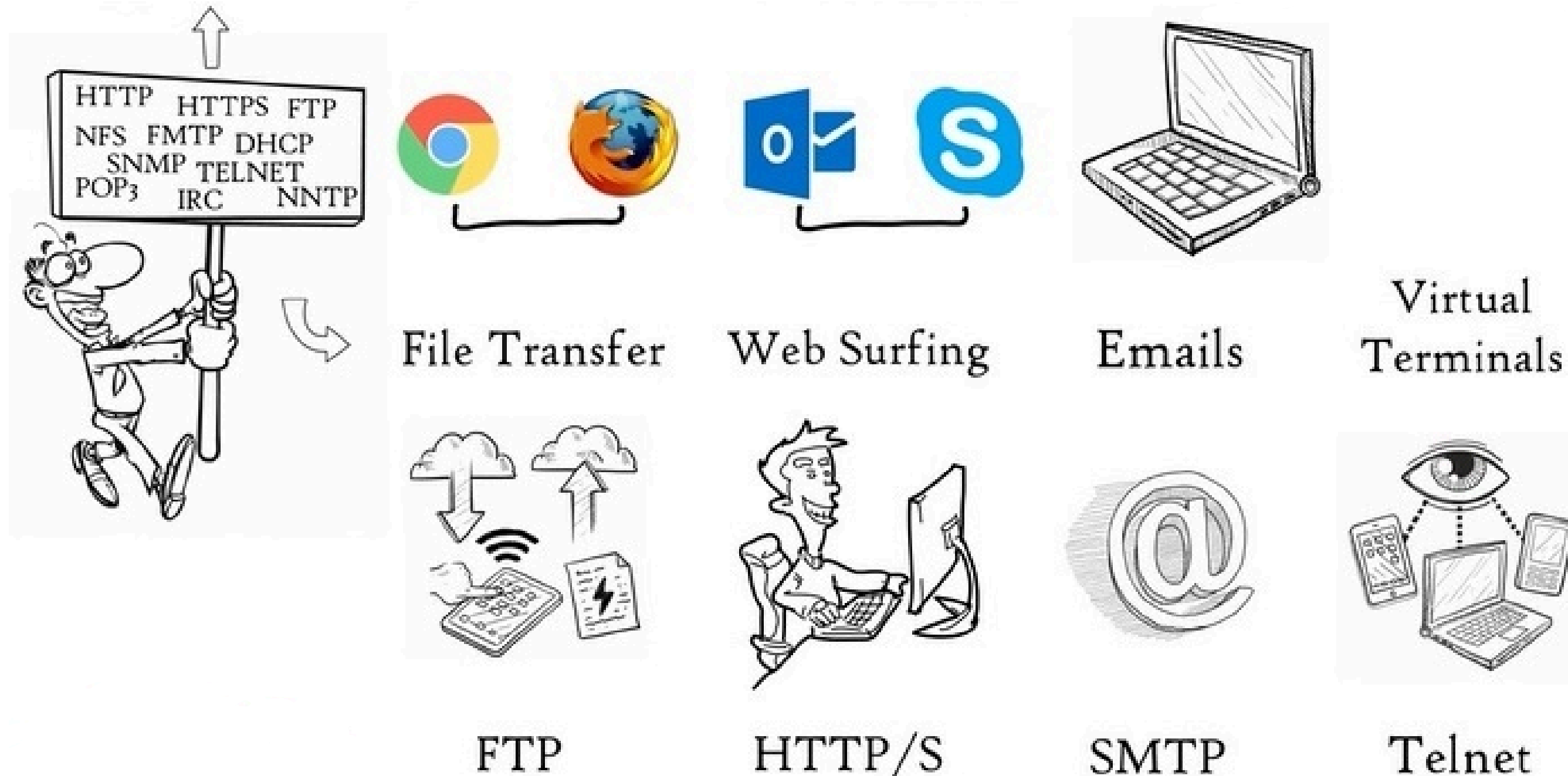
The Application Layer

The Application Layer



The Application Layer (Layer 7) is direct interface between software programs (web browsers and email clients) and the underlying network, interpreting user commands and enabling data exchange across systems.

The Application Layer Protocols



HTTP: The Web's Language

HTTP – The Web's Language

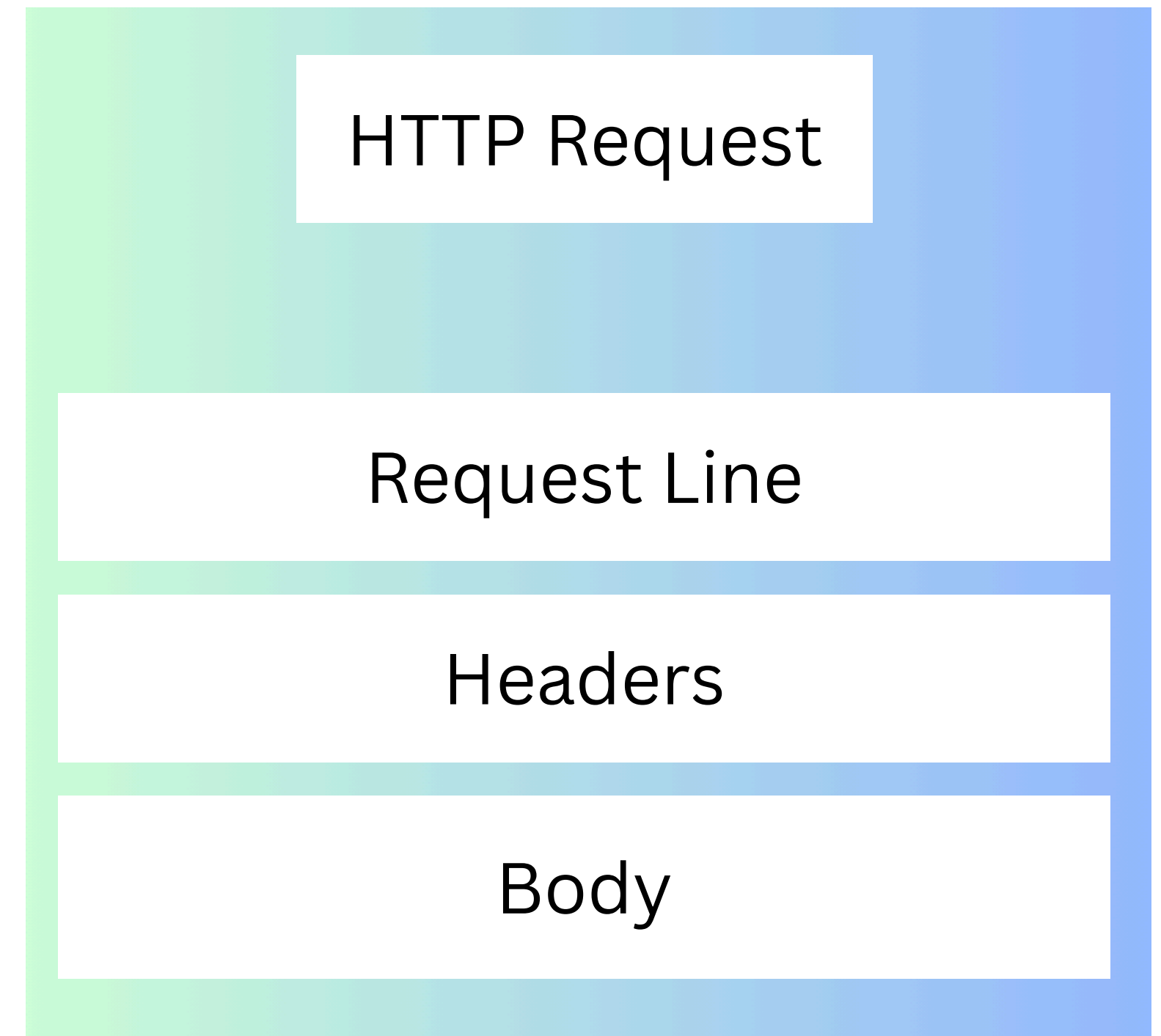
HTTP is a stateless request–response protocol.

- Client (browser) sends a **REQUEST** to a server.
- Server processes and sends back a **RESPONSE**.
- Each exchange is independent the server remembers nothing between requests.

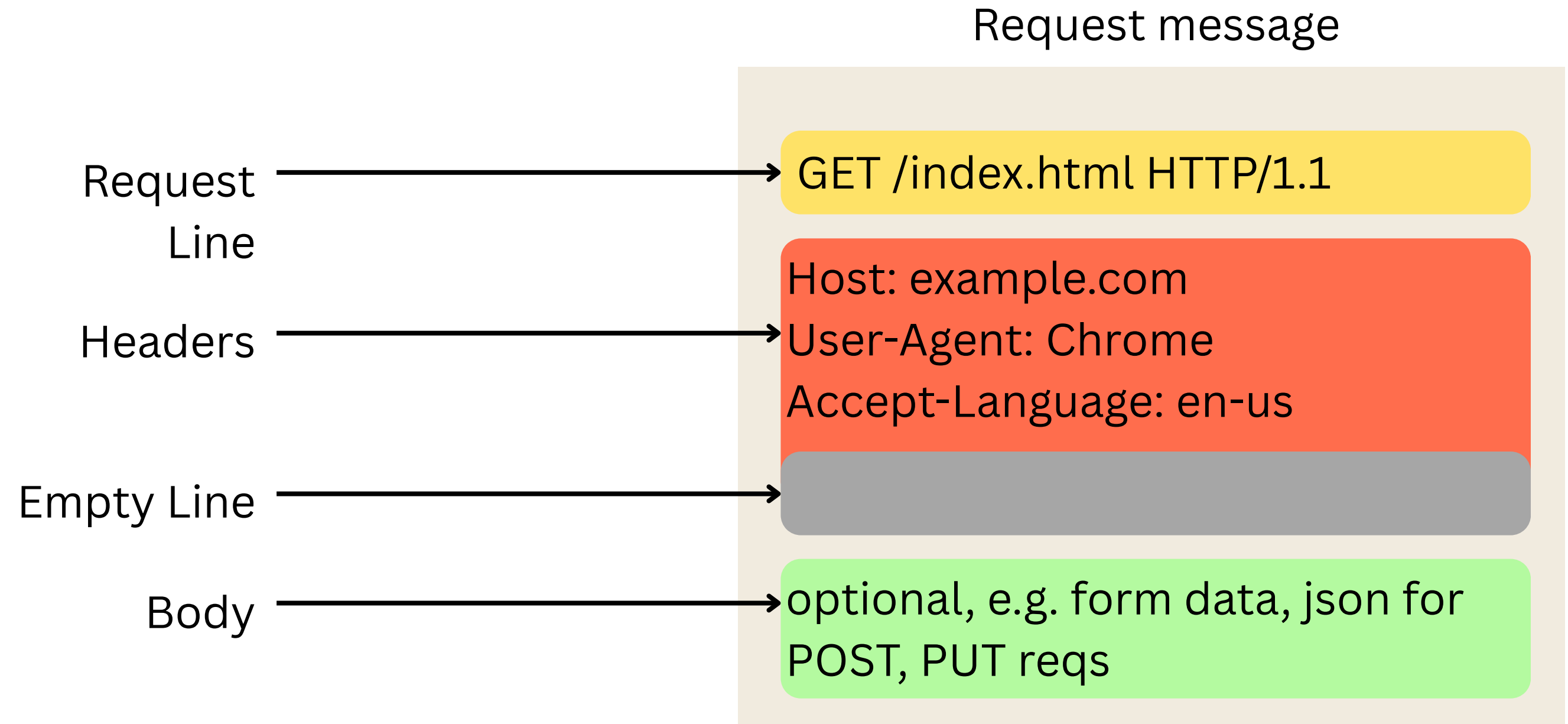


Breakdown of HTTP Requests

- These messages are structured in a way that includes a request/response line, headers, and optionally, a body.



HTTP Requests Breakdown:



HTTP Requests Breakdown:

Request Line:

- The request line contains three parts:
 - **Method:** The type of operation the client wants the server to perform (e.g., GET, POST, PUT, DELETE).
 - **URL:** The resource or endpoint on the server that the client is requesting (e.g., /index.html, /api/data).
 - **HTTP Version:** The version of HTTP being used (e.g., HTTP/1.1 or HTTP/2).
- **Example:**
 - GET /index.html HTTP/1.1

HTTP Requests Breakdown:

Headers:

- Headers provide additional information about the request or about the client's capabilities
- **Common headers include:**
 - **User-Agent:** Information about the client software (browser, app, etc.)
 - **Host:** The domain name of the server
 - **Content-Type:** The type of data the client is sending (for POST or PUT requests)
 - **Accept:** The types of data the client can process (e.g., text/html, application/json)

HTTP Requests Breakdown:

Body (optional):

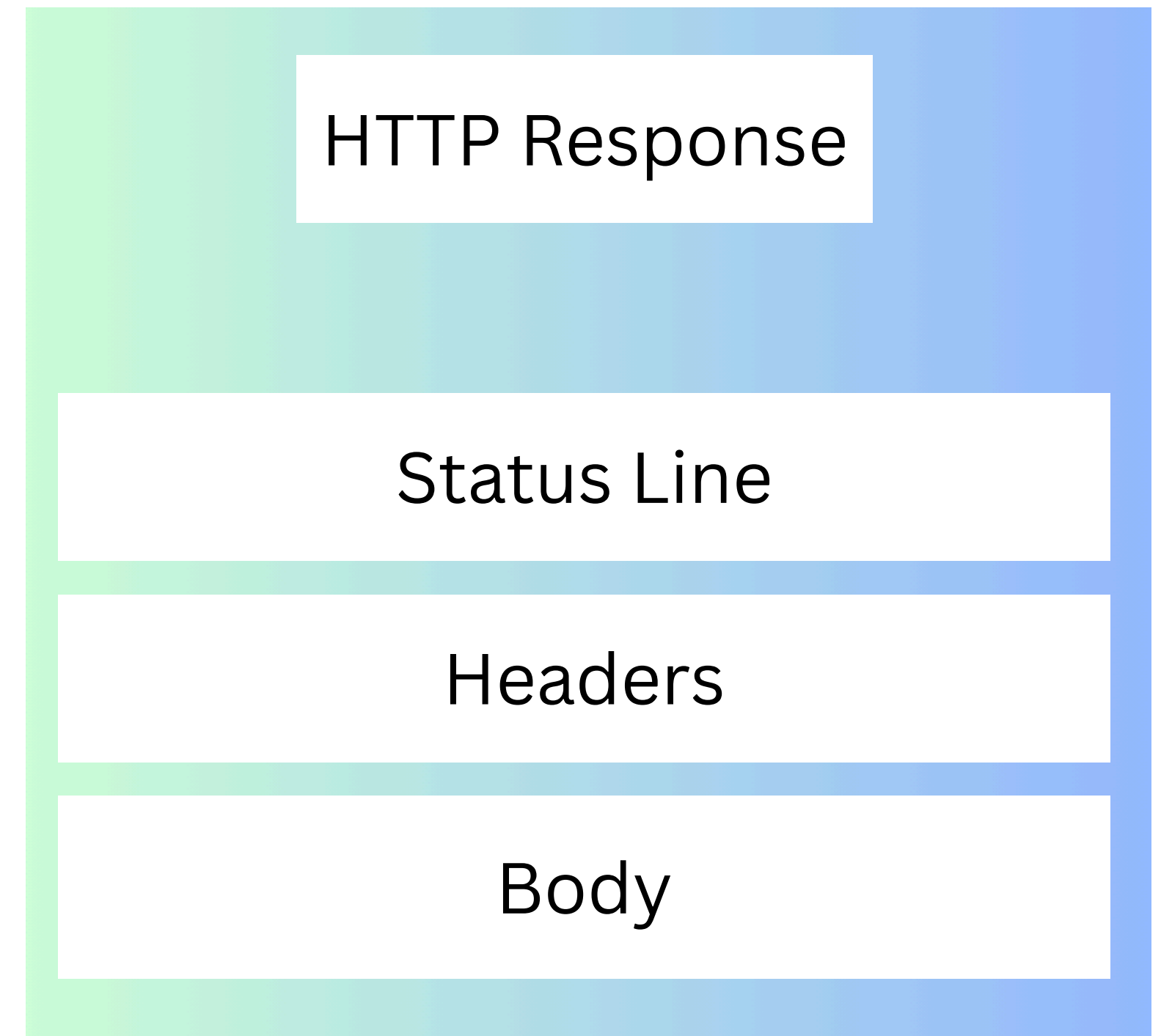
- The body contains the actual data being sent by the client (such as form data, JSON payload, etc.).
- This is only included for certain methods like POST, PUT, or PATCH.

Example (for POST request):

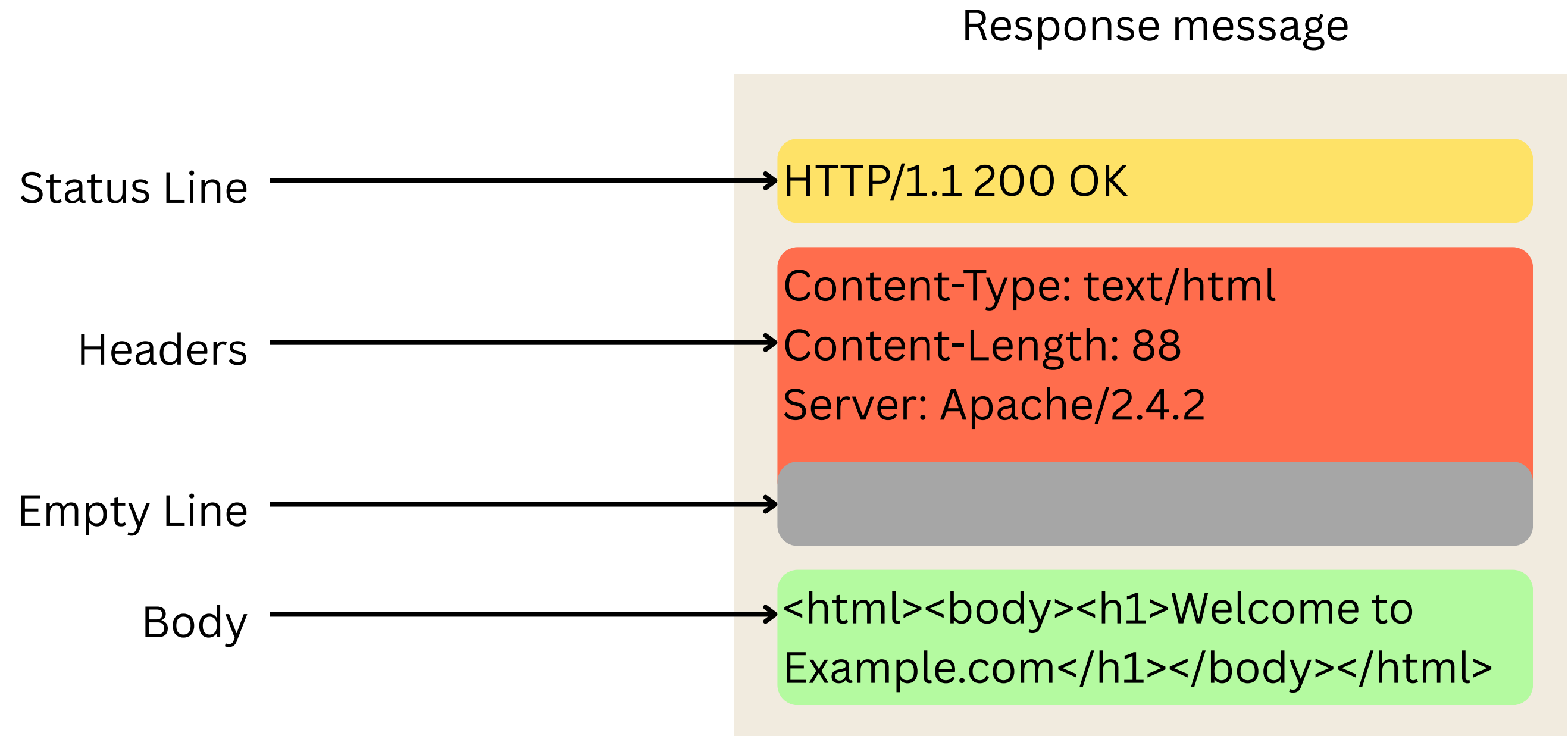
```
{ "username": "user1", "password": "password123" }
```

Breakdown of HTTP Responses

- These messages are structured in a way that includes a request/response line, headers, and optionally, a body.



HTTP Responses Breakdown:



HTTP Status codes:

1. **200 OK** – The request was successful, and the server returned the requested resource.
2. **301 Moved Permanently** – The resource has been permanently moved to a new URL.
3. **302 Found** – Temporary redirection to another URL.
4. **403 Forbidden** – The server understood the request but refuses to authorize it.
5. **404 Not Found** – The server can't find the requested resource.
6. **500 Internal Server Error**

HTTP Status Codes



HTTP Responses Breakdown:

Status Line:

- **The status line consists of:**
 - **HTTP Version:** The HTTP version used in the response.
 - **Status Code:** A 3-digit code indicating the result of the request (e.g., 200, 404, 500).
 - **Status Message:** A short description of the status code (e.g., OK, Not Found, Internal Server Error).

Example:

HTTP/1.1 200 OK

HTTP Response Message Breakdown:

Headers:

- Response headers contain metadata about the response, similar to request headers.
- **Common headers include:**
 - **Content-Type:** The type of data being sent in the body (e.g., text/html, application/json).
 - **Content-Length:** The length of the body in bytes.
 - **Server:** Information about the server software.
 - **Date:** The date and time the response was sent.

Same goes with the BODY for the response. It is similar to request BODY

If HTTP is stateless, how does your shopping cart stay full?

Cookies & Session tokens!

HTTP Generations

The transition from HTTP/1.0 to HTTP/3

The Online Shopping Delivery Problem

Imagine you order a male outfit online: A Shirt, Pants, Shoes, and Socks (these are your HTML, CSS, JS, and Images which we request from server).



How you prefer to have its delivery?

Our preference is answer to the why of HTTP versions.

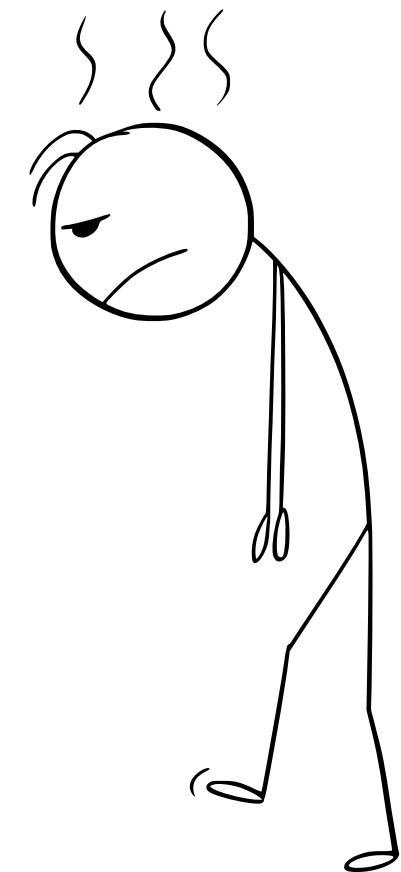
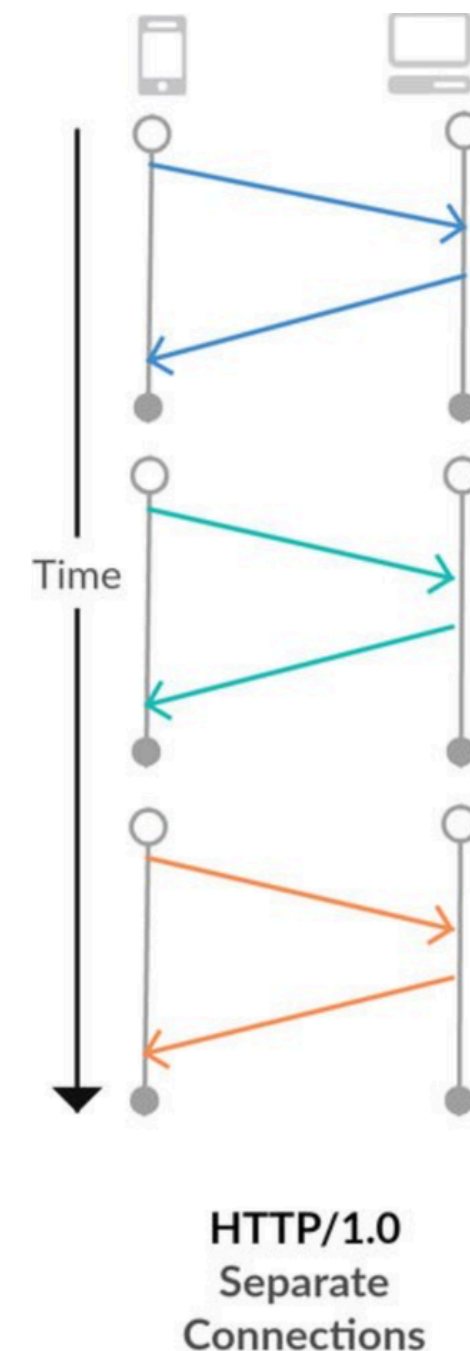
Meet – HTTP/1.0 (1996)

HTTP 1.0 is the truck can only carry one item per trip, and must return to HQ before delivering the next item. A massive waste of time.

- Carries one request at a time.
- Every file (HTML, image, CSS) = new connection.
- **✗** No persistent TCP connection.
- **!** Slow, repetitive, and inefficient.

Quote:

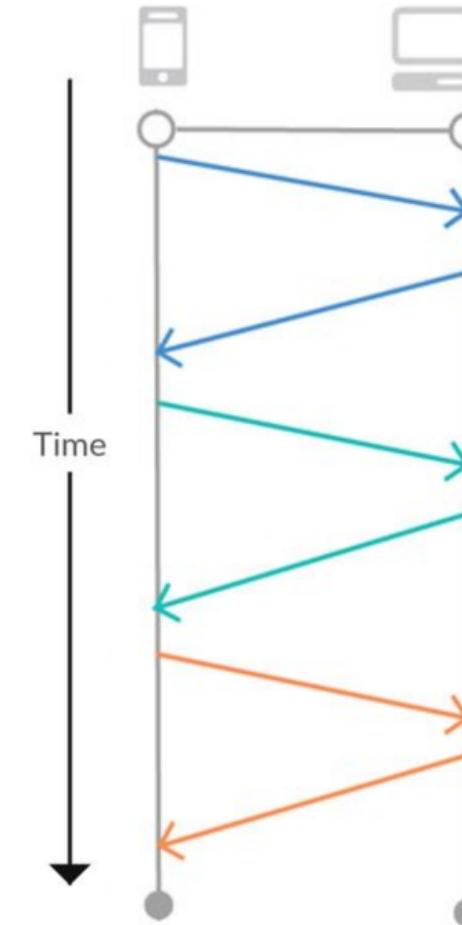
- “I deliver one file at a time. It can get tiring!”



Meet – HTTP/1.1 (1997)

The One-Truck Convoy (Keep-Alive): Truck arrives at your house with all 4 items but Unloads/Deliver it One by One.

- **Introduced:**
 - Keep-Alive connections
 - Caching and Host Headers
- Improved performance but had head-of-line blocking.



HTTP/1.1
Persistent
Connections,
Request Queuing



<https://captive.apple.com>

Meet – HTTP/1.1 (1997)

ONE CONNECTION • responses must exit in order (FIFO) →

slow image



CSS

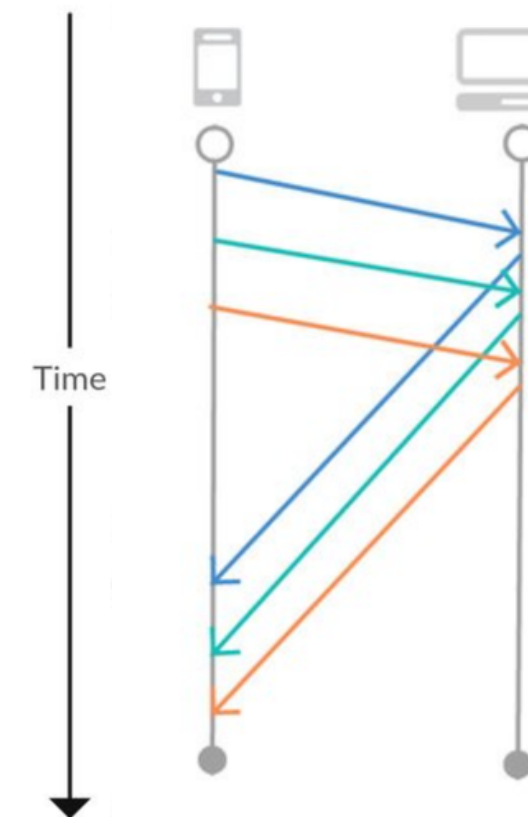
JS

icon

← all blocked, even though they're ready

HTTP/1.1 Pipelining (optional)

- Responses must arrive in the same order as requests (**FIFO - First In, First Out**)
- Only safe methods can be pipelined (**GET, HEAD, OPTIONS**)
- No POST/PUT - these might have side effects as they alter the state of destination.



Quote:

“Multiple requests can be made without waiting for response for previous request”

HTTP/1.1
Pipelining,
Response Queuing,
Head-of-line
blocking

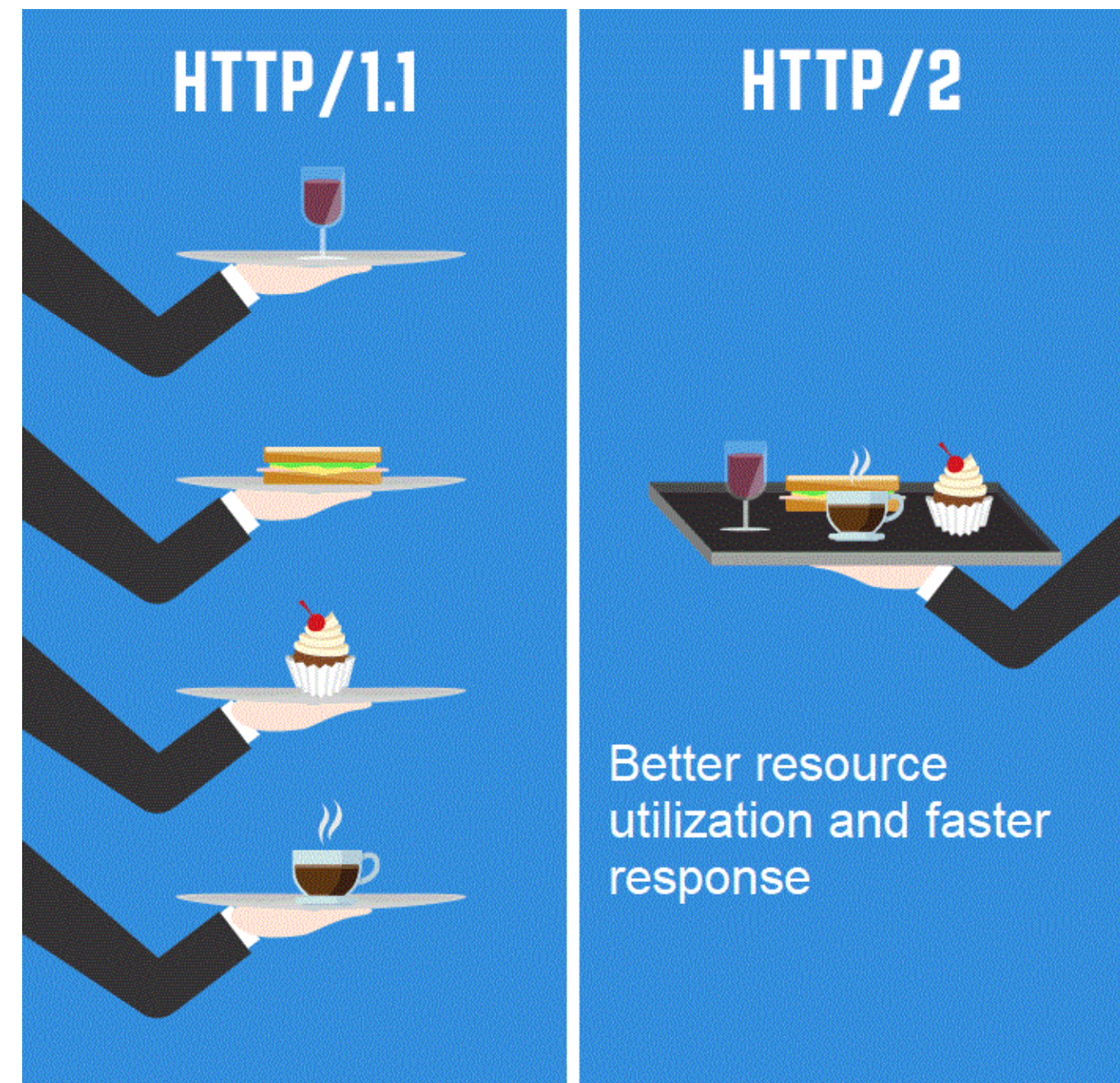


Meet – HTTP/2 (2015)

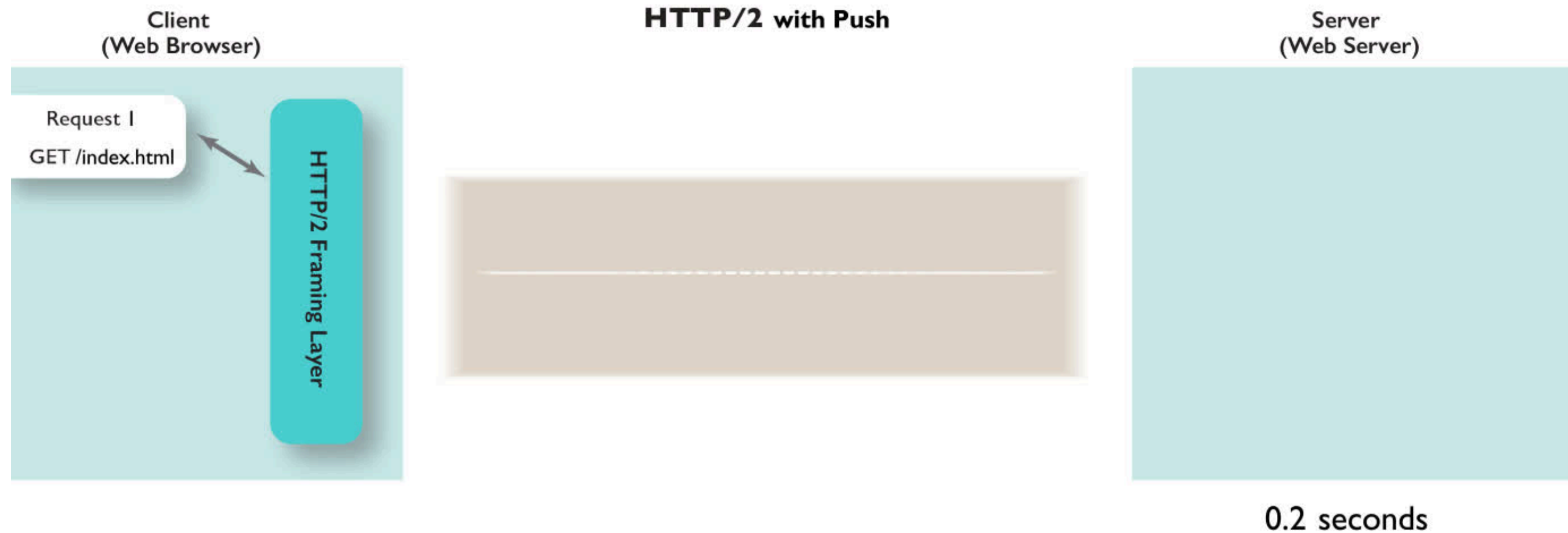
Instead of serving your meal one by one. I will Prefer to serve you entire tray and now you choose what to eat.

- HTTP/2 is the second major version of HTTP, aimed at enhancing data transfer performance between clients and servers.
- It improves upon HTTP/1.1 by addressing its limitations, particularly in terms of performance.

https://en.wikipedia.org/wiki/HTTP_pipelining



HTTP/2



Binary Protocol | Multiplexing | Server Push

<https://manningbooks.medium.com/http-1-1-vs-http-2-vs-http-2-with-push-91f7d497ddbe>

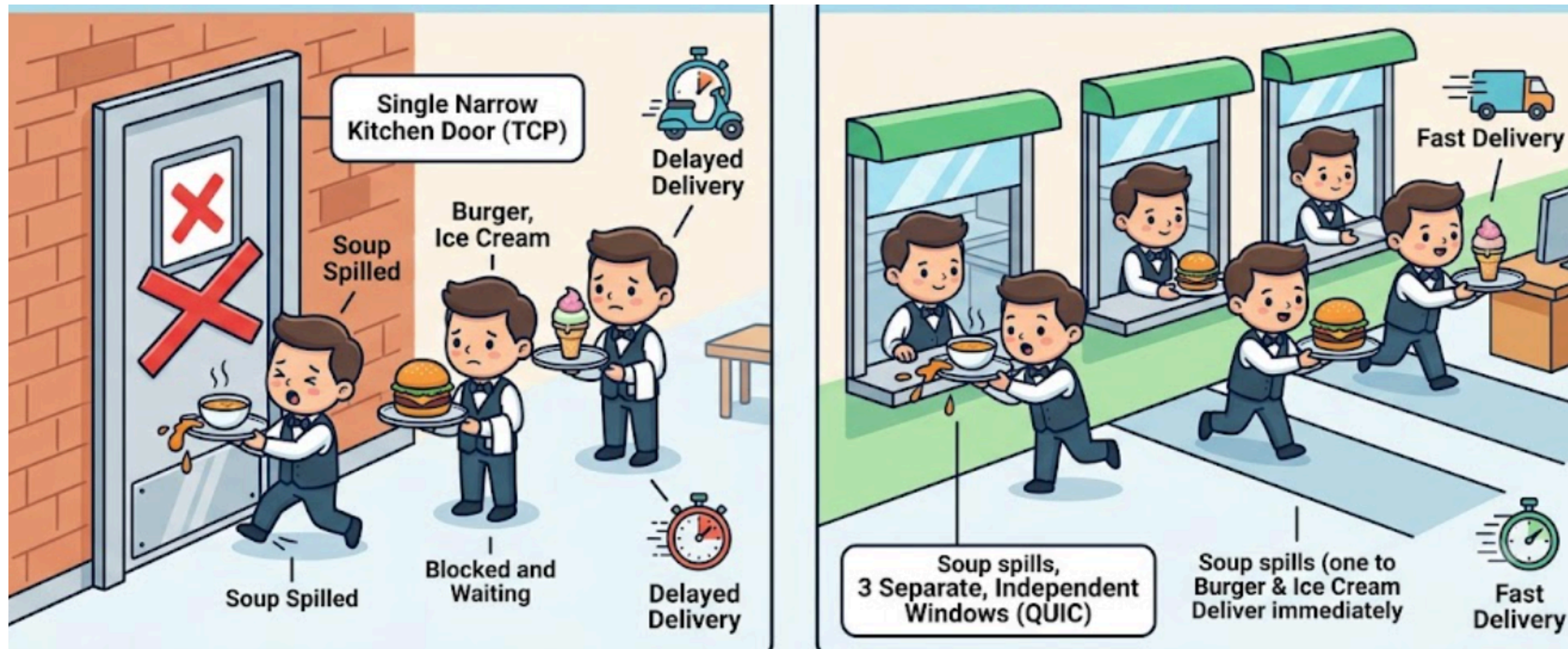
HTTP/2 Solutions

HTTP/2 Features:

- Uses single TCP connection.
- **Binary Protocol:** Uses a binary format for message encoding, unlike the text-based HTTP. That is the framing layer.
- **Multiplexing:** Enables simultaneous requests and responses over one connection.
- **Server Push:** Allows servers to send resources to clients proactively without explicit requests.



What if a service like this?

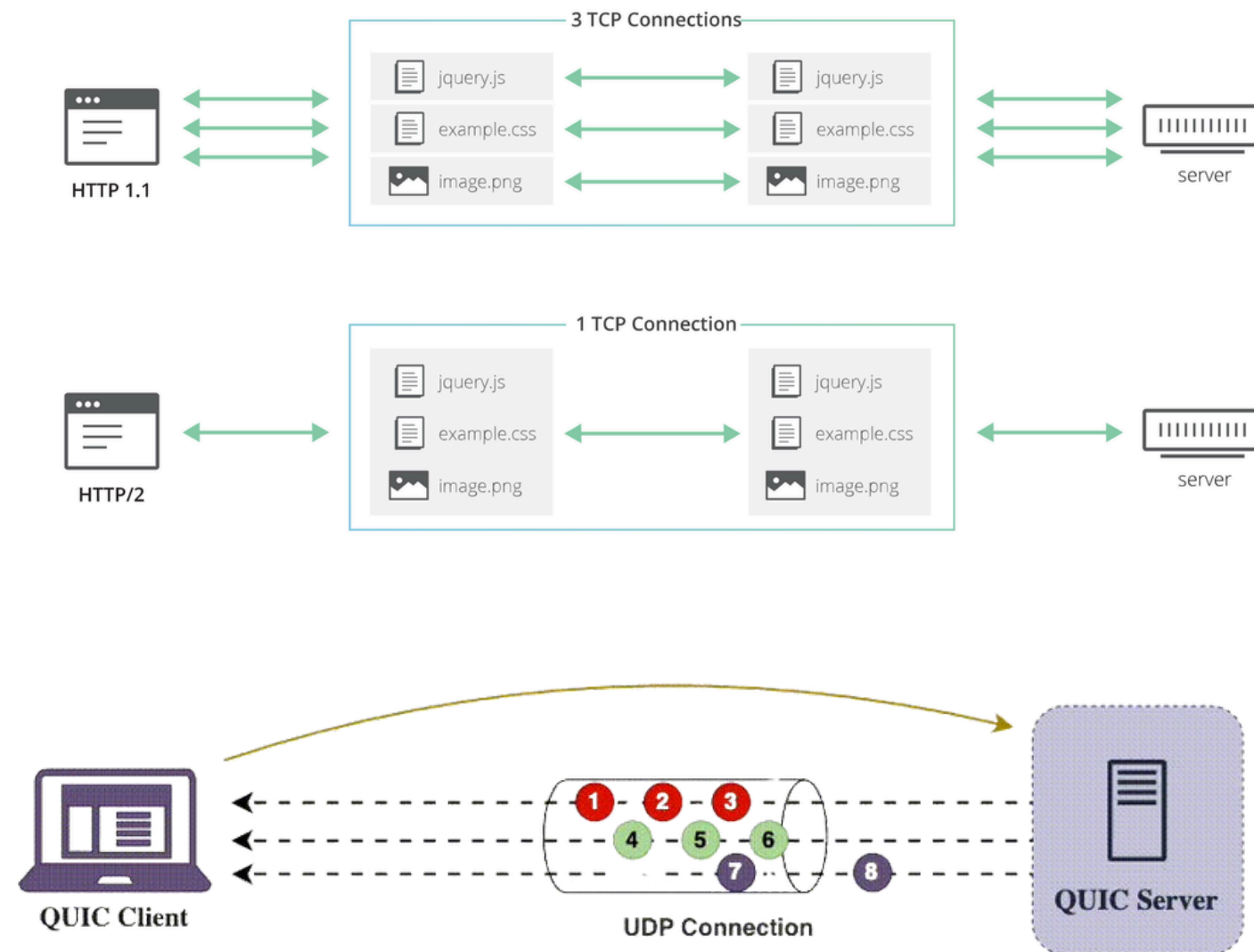


You order 3 item at restro and their entire staff started serving only you and everything you ordered is having their seperate kitchen window and dedicated staffs.

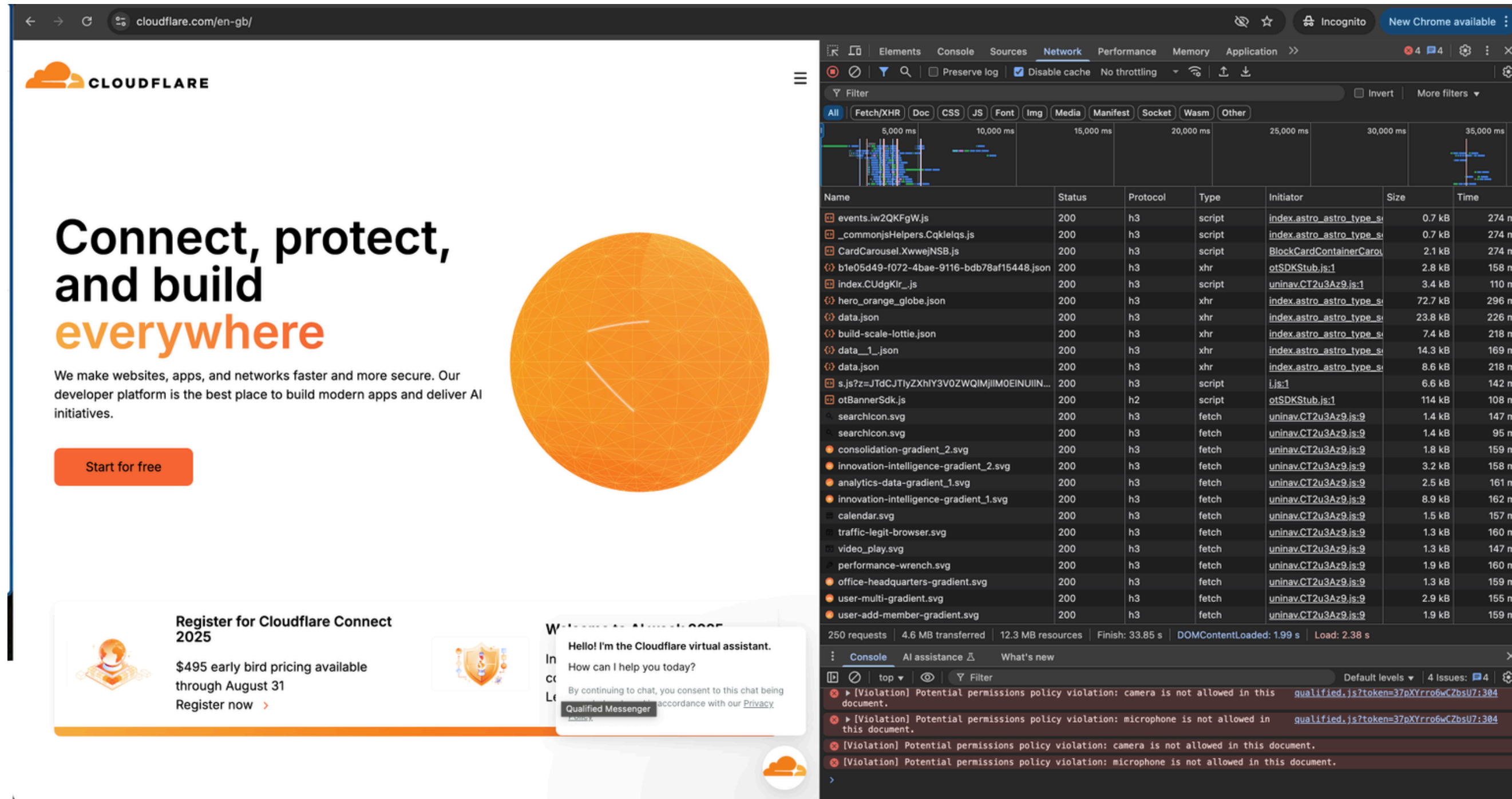
HTTP/3 using QUIC (June 2022)

HTTP/3 Features:

- Latest version of the HTTP protocol
- Built on QUIC, a **UDP-based** transport protocol instead of **TCP** developed by Google
- Multiplexed streams without blocking
- Designed to solve head-of-line blocking



HTTP/3 using QUIC (June 2022)



The screenshot shows the Cloudflare website with the headline "Connect, protect, and build everywhere". The Chrome DevTools Network tab is open, displaying a list of resources loaded over HTTP/3. The resources include various JavaScript files, JSON files, and SVG images, all loaded via the h3 protocol. The console shows several warnings related to permissions policy violations for camera and microphone access.

Name	Status	Protocol	Type	Initiator	Size	Time
events.iw2QKFgW.js	200	h3	script	index.astro_astro_type_s	0.7 kB	274 ms
_commonjsHelpers.Cqklelqs.js	200	h3	script	index.astro_astro_type_s	0.7 kB	274 ms
CardCarousel.XwwejNSB.js	200	h3	script	BlockCardContainerCarou	2.1 kB	274 ms
b1e05d49-f072-4bae-9116-bdb78af15448.json	200	h3	xhr	otSDKStub.js:1	2.8 kB	158 ms
index.CUdgKir_.js	200	h3	script	uninav.CT2u3Az9.js:1	3.4 kB	110 ms
hero_orange_globe.json	200	h3	xhr	index.astro_astro_type_s	72.7 kB	296 ms
data.json	200	h3	xhr	index.astro_astro_type_s	23.8 kB	226 ms
build-scale-lottie.json	200	h3	xhr	index.astro_astro_type_s	7.4 kB	218 ms
data_1.json	200	h3	xhr	index.astro_astro_type_s	14.3 kB	169 ms
data.json	200	h3	xhr	index.astro_astro_type_s	8.6 kB	218 ms
s.js?z=JTdCJTlyZXh1Y3V0ZWQIMjllMOEINUIIN...	200	h3	script	i.js:1	6.6 kB	142 ms
otBannerSdk.js	200	h2	script	otSDKStub.js:1	114 kB	108 ms
searchcon.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.4 kB	147 ms
searchcon.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.4 kB	95 ms
consolidation-gradient_2.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.8 kB	159 ms
innovation-intelligence-gradient_2.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	3.2 kB	158 ms
analytics-data-gradient_1.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	2.5 kB	161 ms
innovation-intelligence-gradient_1.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	8.9 kB	162 ms
calendar.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.5 kB	157 ms
traffic-legit-browser.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.3 kB	160 ms
video_play.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.3 kB	147 ms
performance-wrench.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.9 kB	160 ms
office-headquarters-gradient.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.3 kB	159 ms
user-multi-gradient.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	2.9 kB	155 ms
user-add-member-gradient.svg	200	h3	fetch	uninav.CT2u3Az9.js:9	1.9 kB	159 ms

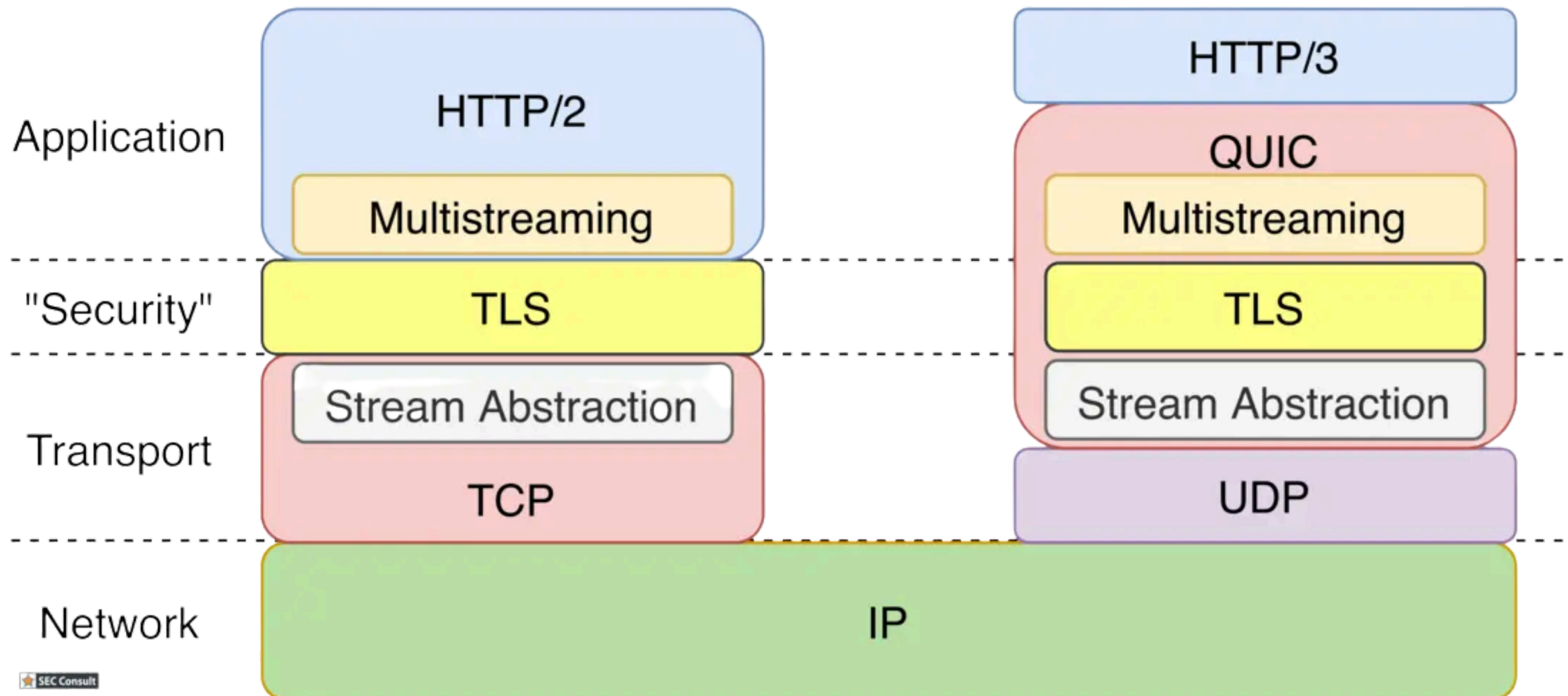
250 requests | 4.6 MB transferred | 12.3 MB resources | Finish: 33.85 s | DOMContentLoaded: 1.99 s | Load: 2.38 s

Console: 4 Issues: 4

- [Violation] Potential permissions policy violation: camera is not allowed in this document.
- [Violation] Potential permissions policy violation: microphone is not allowed in this document.
- [Violation] Potential permissions policy violation: camera is not allowed in this document.
- [Violation] Potential permissions policy violation: microphone is not allowed in this document.

cloudflare.com

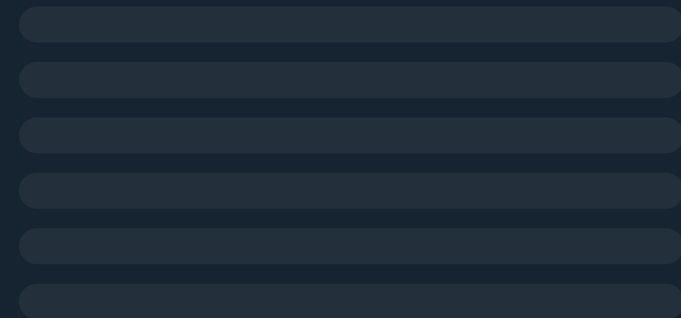
HTTP/3 using QUIC (June 2022)



Transition

HTTP/1.1

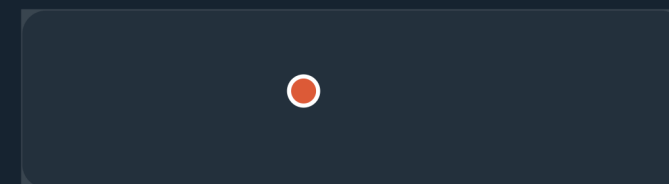
6 separate TCP connections
to fake parallelism



Costly: 6× setup, 6× overhead

HTTP/2

Many streams multiplexed
on one TCP connection



1 lost TCP
segment → — all
streams
stall

App-level HoL fixed — TCP-
level remains

HTTP/3

streams
QUIC over UDP — truly
independent

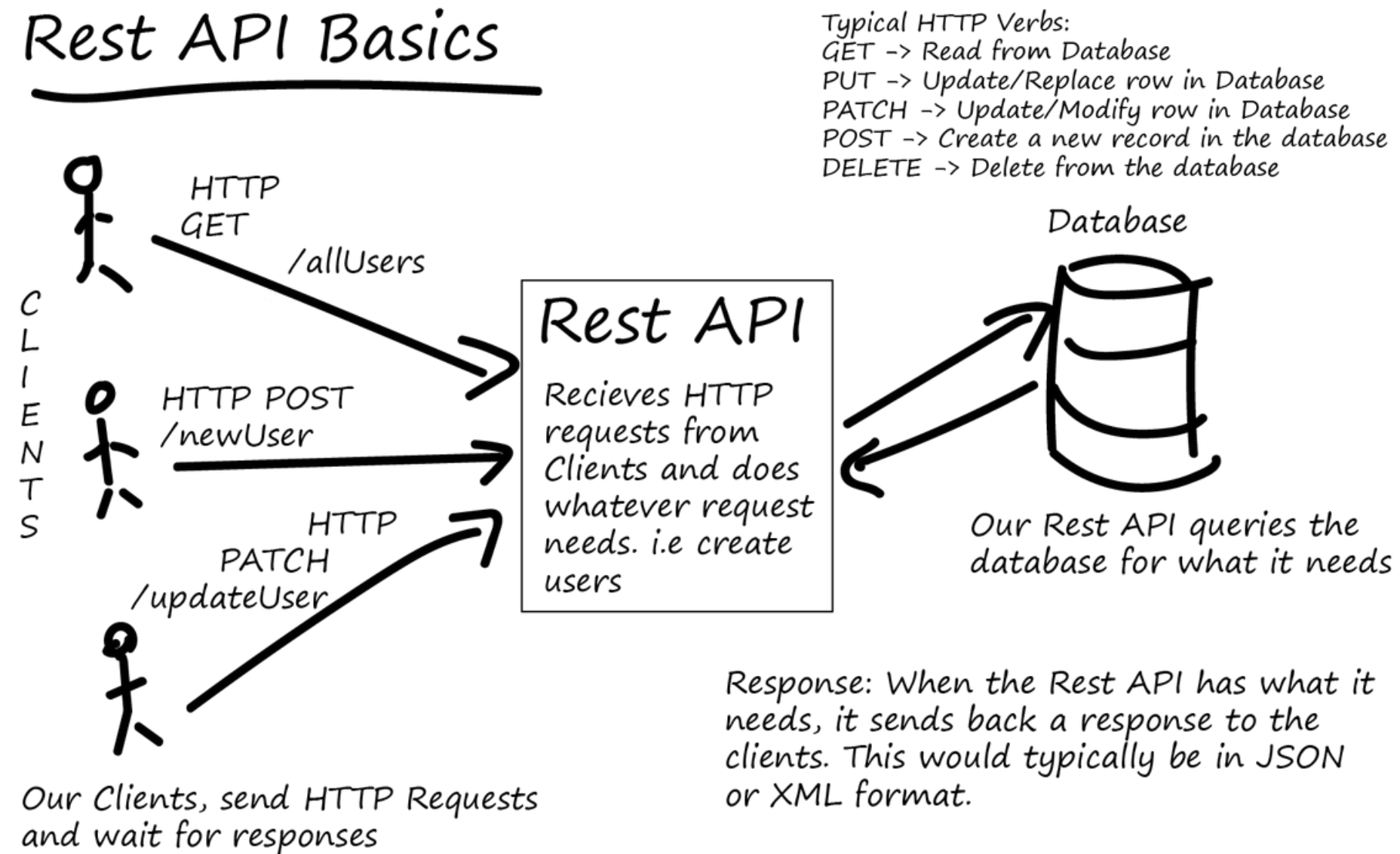


Loss is isolated to its
own stream

**We understand the Client-Server model,
where a browser requests a page from a
server, But what happens when Application
need to communicate with another
Applications?**

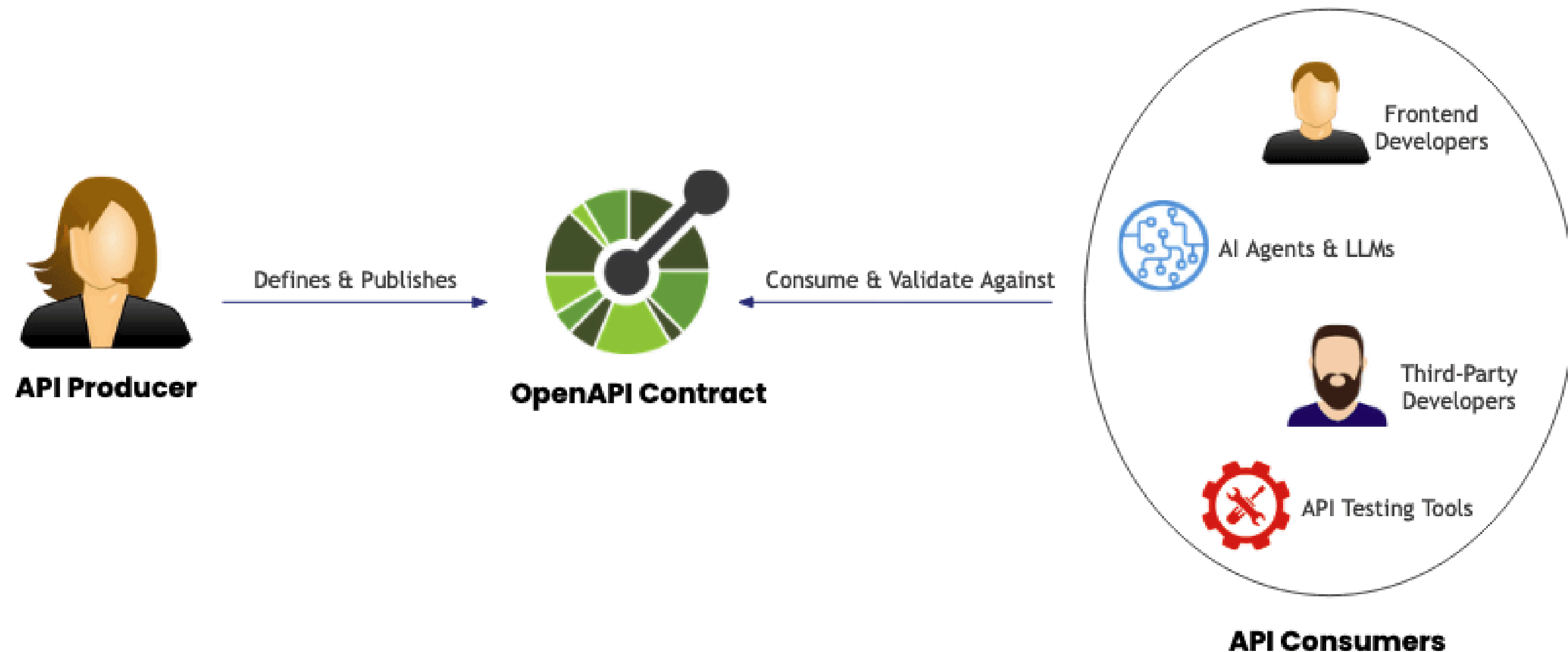
REST APIs

Rest API Basics



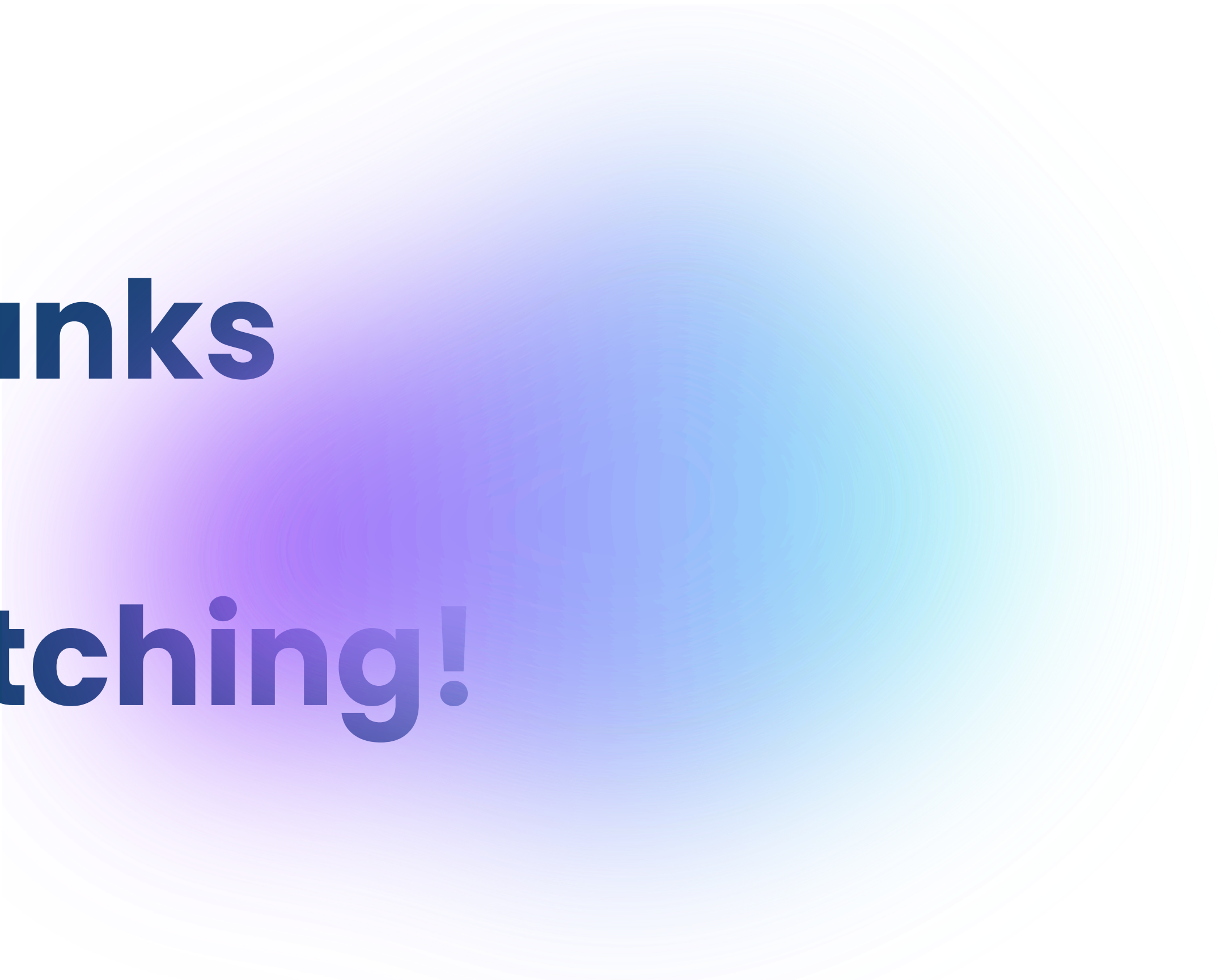
- A Q4 2024 analysis estimating 83% of all public APIs use REST architecture

Why use OpenAPI?



<https://petstore3.swagger.io>

Please fill the feedback form.



**Thanks
for
watching!**